

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Тема: Обработка таблиц

Вариант № 5

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: ***<Фамилия И.О.>***

2025 г.

Описание задачи

Создать таблицу, содержащую не менее 40-ка записей. Упорядочить данные в ней по возрастанию ключей, двумя алгоритмами сортировки, где ключ – любое невариантное поле (по выбору программиста), используя: а) саму таблицу, б) массив ключей. (Возможность добавления и удаления записей в ручном режиме обязательна). Осуществить поиск информации по варианту. Ввести репертуар театров, содержащий: название театра, спектакль, диапазон цены билета, тип спектакля (1: пьеса, 2: драма, 3: комедия, 4: сказка - возраст (3+, 10+, 16+), 5: музыкальный – композитор, страна, тип (балет, опера, мюзикл), возраст (3+, 10+, 16+), продолжительность). Вывести список всех балетов для детей указанного возраста с продолжительностью меньше указанной.

Техническое задание

Описание исходных данных и результатов

Исходные данные:

- Название театра (строка до 100 символов)
- Название спектакля (строка до 100 символов)
- Диапазон цен: минимальная и максимальная цена (целые положительные числа)
- Тип спектакля:
  - 0 - Пьеса (драма, комедия, сказка)
  - 1 - Музыкальный (балет, опера, мюзикл)

Для пьес:

- Тип пьесы: 0-драма, 1-комедия, 2-сказка
- Для сказок - возрастное ограничение (3+, 10+, 16+)

Для музыкальных спектаклей:

- Композитор (строка)
- Страна (строка)
- Тип: 0-балет, 1-опера, 2-мюзикл
- Возрастное ограничение (3+, 10+, 16+)
- Продолжительность в минутах (целое положительное число)

Результаты:

1. Ввод данных: Пользователь может ввести данные о театральных представлениях, указывая все необходимые атрибуты.
2. Вывод данных: Просмотр всех введенных данных в табличном формате.
3. Вывод таблицы ключей: Создание и просмотр таблицы ключей на основе названий спектаклей.
4. Добавление записи: Добавление новых записей о театральных представлениях.
5. Удаление записи: Удаление записей по названию спектакля.
6. Сортировка таблицы: Два варианта сортировки данных по названию спектакля - пузырьковая и быстрая.

7. Сортировка таблицы ключей: Сортировка таблицы ключей двумя алгоритмами.
8. Вывод данных по ключам: Просмотр данных, отсортированных по таблице ключей.
9. Поиск балетов: Вывод списка балетов для детей указанного возраста с продолжительностью меньше указанной.
10. Сравнение эффективности: Анализ времени выполнения сортировок и использования памяти.

Описание задачи, реализуемой программой

Программа реализует систему управления данными о театральных спектаклях с возможностью:
- Ввода и валидации данных
- Хранения информации в структурах с вариативной частью
- Сортировки по названию спектакля
- Поиска балетов по возрастному ограничению и продолжительности
- Сравнения эффективности различных алгоритмов обработки

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка возникает при попытке работы с пустой таблицей (вывод, сортировка, создание ключей).
2. Ошибка открытия файла при генерации тестовых данных.
3. Ошибка переполнения таблицы при добавлении записей (более 40 записей).
4. Ошибка неверного индекса при удалении записи.
5. Ошибка некорректного ввода данных (неправильный формат, отрицательные значения).
6. Ошибка неверного формата возрастной категории.
7. Ошибка отсутствия записи при поиске.

Описание внутренних структур данных

Перечисление для возрастных ограничений:

```
enum AGE_LIMIT {AGE_3, AGE_10, AGE_16, AGE_ERROR};
```

значение 0 - соответствует возрастной категории "3+"
значение 1 - соответствует возрастной категории "10+"
значение 2 - соответствует возрастной категории "16+"
значение 3 - код ошибки при неверном формате возраста

Структура для пьесы:

```
struct Play {
    int play_type;
    enum AGE_LIMIT age;
};
```

1. play_type - определяет жанр пьесы:

- 0: Драма (не требует возрастного ограничения)
- 1: Комедия (не требует возрастного ограничения)
- 2: Сказка (требует указания возрастного ограничения)

2. age - возрастное ограничение, используется только когда play_type == 2 (сказка)

Структура для музыкального спектакля:

```
struct Musical {
    char composer[MAX_LEN];
    char country[MAX_LEN];
    int musical_type;
    enum AGE_LIMIT age;
    int duration;
};
```

3. composer - имя композитора (обязательное поле)
4. country - страна создания спектакля (обязательное поле)
5. musical_type - тип музыкального произведения:
   - 0: Балет (используется в поиске по варианту)
   - 1: Опера
   - 2: Мюзикл
6. age - возрастное ограничение (обязательное для всех музыкальных спектаклей)
7. duration - продолжительность в минутах (целое положительное число)

Основная структура спектакля с вариантами:

```
struct Spectacle {
    char theatre_name[MAX_LEN];
    char spec_name[MAX_LEN];
    int min_price;
    int max_price;
    int type;

    union {
        struct Play play;
        struct Musical musical;
    };
};
```

1. theatre_name - строка длиной до 100 символов, содержит название театра
2. spec_name - строка длиной до 100 символов, содержит название спектакля (используется как ключ для сортировки)
3. min_price - целое положительное число, минимальная стоимость билета

4. max_price - целое положительное число, максимальная стоимость билета (должна быть >= min_price)
5. type - определяет тип спектакля:
   - 0: Пьеса - активирует использование union.play
   - 1: Музыкальный спектакль - активирует использование union.musical

Структура для таблицы ключей:

```
struct KeyRecord {
   char key_value[MAX_LEN];
   int original_index;
};
```

1. key_value - копия названия спектакля из основной таблицы, используется для сортировки
2. original_index - позиция записи в исходной таблице, позволяет найти полную информацию о спектакле после сортировки ключей

Общий алгоритм программы:

1. Инициализация: Объявление массивов для основной таблицы и таблицы ключей.
2. Основной цикл программы:
   1. Вывод меню с доступными операциями
   2. Ожидание ввода выбора пользователя
   3. Выполнение выбранной операции
3. Операции меню:
   1: Добавление нового спектакля с валидацией ввода
   2: Удаление спектакля по названию
   3: Просмотр всей таблицы
   4: Создание таблицы ключей
   5: Просмотр таблицы ключей
   6: Сортировка основной таблицы пузырьком
   7: Сортировка основной таблицы быстрой сортировкой
   8: Сортировка таблицы ключей пузырьком
   9: Сортировка таблицы ключей быстрой сортировкой
   10: Просмотр таблицы через ключи
   11: Поиск балетов по критериям
   12: Генерация тестовых данных из файла
   13: Сравнение эффективности сортировок
   0: Выход из программы

Описание алгоритмов функций

print_table(spectacle_table, spectacle_count)

- Если количество записей равно 0, вывести сообщение "Пустая таблица"
- Для каждого элемента таблицы от первого до последнего вызвать функцию print_spectacle()

- Завершить выполнение

add_spectacle(spectacle_table, spectacle_count)

- Проверить, не превышен ли максимальный размер таблицы
- Вызвать функцию input_spectacle() для ввода данных новой записи
- Увеличить счетчик записей на 1
- Завершить выполнение

delete_spectacle(index, spectacle_table, spectacle_count)

- Проверить корректность индекса
- Сдвинуть все элементы, начиная с позиции index+1, на одну позицию влево
- Уменьшить счетчик записей на 1
- Вывести сообщение об успешном удалении

find_spectacle_by_name(name, spectacle_table, spectacle_count)

- Для каждого элемента таблицы сравнить название спектакля с искомым
- При нахождении совпадения вернуть индекс элемента
- Если совпадений нет, вернуть NOT_FOUND

find_ballet(spectacle_table, spectacle_count)

- Ввести возрастную категорию с проверкой корректности формата
- Ввести максимальную продолжительность с проверкой положительности значения
- Для каждого элемента таблицы проверить:
  - Является ли спектакль музыкальным
  - Является ли тип музыкального спектакля балетом
  - Соответствует ли возрастная категория заданной
  - Меньше ли продолжительность заданной
- Вывести найденные спектакли или сообщение об отсутствии результатов

generate_test_data(spectacle_table, spectacle_count)

- Открыть файл "data.txt" для чтения
- Проверить успешность открытия файла
- Читать записи из файла пока не достигнут конец файла или максимальный размер таблицы
- Для каждой успешно прочитанной записи увеличить счетчик
- Закрыть файл и вывести количество добавленных записей

input_spectacle(s)

- Ввести название театра с проверкой непустоты строки
- Ввести название спектакля с проверкой непустоты строки
- Ввести минимальную и максимальную цену с проверкой положительности
- Ввести тип спектакля с проверкой допустимых значений (0 или 1)
- Если тип - пьеса (0):
  - Ввести тип пьесы с проверкой допустимых значений (0-2)
  - Если тип пьесы - сказка (2), ввести возрастную категорию
- Если тип - музыкальный (1):
  - Ввести композитора и страну с проверкой непустоты строк
  - Ввести тип музыкального спектакля с проверкой допустимых значений (0-2)
  - Ввести возрастную категорию
  - Ввести продолжительность с проверкой положительности

  print_spectacle(s, index)

- Вывести номер записи, название театра и спектакля
- Вывести диапазон цен
- Вывести тип спектакля
- Если тип - пьеса, вывести жанр и для сказок - возрастную категорию
- Если тип - музыкальный, вывести композитора, страну, тип, возрастную категорию и продолжительность

  age_limit_from_string(s)

- Удалить символ новой строки из входной строки
- Сравнить строку с "16+", "10+", "3+"
- Вернуть соответствующее значение перечисления AGE_LIMIT
- Если строка не соответствует ни одному варианту, вернуть AGE_ERROR

  bubble_sort_spectacles(spectacle_table, spectacle_count)

- Для i от 0 до spectacle_count-2
  - Для j от 0 до spectacle_count-i-2
    - Если название спектакля j больше названия спектакля j+1
      - Поменять местами записи j и j+1

  bubble_sort_keys(key_table, key_count)

- Для i от 0 до key_count-2
  - Для j от 0 до key_count-i-2
    - Если ключ j больше ключа j+1
      - Поменять местами ключи j и j+1

  quick_sort_spectacles(left, right, spectacle_table, spectacle_count)

- Если left >= right, завершить рекурсию
- Выбрать опорный элемент из середины диапазона
- Разделить массив на две части относительно опорного элемента
- Рекурсивно вызвать функцию для левой и правой частей

quick_sort_keys(left, right, key_table, key_count)

- Если left >= right, завершить рекурсию
- Выбрать опорный ключ из середины диапазона
- Разделить массив ключей на две части относительно опорного ключа
- Рекурсивно вызвать функцию для левой и правой частей

create_key_table(spectacle_table, spectacle_count, key_table, key_count)

- Проверить, что таблица не пуста
- Установить количество ключей равным количеству записей
- Для каждой записи в основной таблице:
  - Скопировать название спектакля в ключ
  - Сохранить исходный индекс записи

print_key_table(key_count, key_table)

- Если таблица ключей пуста, вывести сообщение
- Для каждого ключа вывести его значение и исходный индекс

print_table_by_keys(spectacle_table, key_table, key_count)

- Если таблица ключей пуста, вывести сообщение
- Для каждого ключа в отсортированном порядке:
  - Получить исходный индекс из ключа
  - Вывести запись из основной таблицы по этому индексу

compare_efficiency(spectacle_table, spectacle_count, key_table, key_count)

- Создать копии основной таблицы и таблицы ключей
- Измерить время выполнения четырех видов сортировки:
  - Пузырьковая сортировка основной таблицы
  - Быстрая сортировка основной таблицы
  - Пузырьковая сортировка таблицы ключей
  - Быстрая сортировка таблицы ключей
- Вычислить эффективность в процентах
- Сравнить использование памяти двумя подходами
- Вывести результаты сравнения

Тесты

Позитивные тесты

Номер теста	Описание	Ожидаемый результат
1	Добавление записи и вывод.	Успешное добавление новой записи и вывод на экран.
2	Добавление нескольких записей и вывод.	Успешное добавление нескольких записей и вывод на экран.
3	Чтение данных из файла и вывод.	Успешное считывание данных из файла и вывод на экран.
4	Сортировка и вывод.	Успешная сортировка данных и вывод в правильном порядке.
5	Удаление записи и вывод.	Успешное удаление выбранной записи и вывод обновленной таблицы.
6	Сортировка таблицы ключей и вывод.	Успешная сортировка таблицы ключей и вывод в правильном порядке.
7	Вывод данных по таблице ключей.	Успешный вывод данных о театральных представлениях по таблице ключей.
8	Поиск балетов по критериям.	Успешный поиск и вывод балетов, соответствующих заданным параметрам.
9	Создание таблицы ключей.	Успешное создание таблицы ключей на основе основной таблицы.

10	Сравнение эффективности сортировок.	Успешное сравнение времени выполнения и использования памяти разных алгоритмов.

Негативные тесты

Номер теста	Описание	Ожидаемый результат
1	Попытка добавления записи с недопустимой ценой билета (отрицательной).	Заново ввести цену билета
2	Попытка добавления записи с недопустимым типом спектакля (не в диапазоне 0-1).	Заново ввести тип спектакля
3	Попытка добавления записи с недопустимым возрастным диапазоном (не "3+", "10+" или "16+").	Заново ввести возрастной диапазон
4	Попытка добавления записи с недопустимым типом музыкального спектакля (не в диапазоне 0-2).	Заново ввести тип музыкального спектакля
5	Попытка добавления записи с недопустимой продолжительностью (отрицательной).	Заново ввести продолжительность
6	Попытка добавления записи при достижении максимального количества записей	Сообщение об ошибке: Таблица переполнена

7	Попытка удаления записи с недопустимым индексом	Сообщение об ошибке: Неверный индекс
8	Попытка удаления записи с несуществующим названием	Сообщение об ошибке: Спектакль не найден
9	Попытка чтения данных из несуществующего файла.	Сообщение об ошибке: Ошибка: не удалось открыть файл data.txt
10	Попытка работы с пустой таблицей (вывод, сортировка).	Сообщение об ошибке: Пустая таблица
11	Попытка работы с пустой таблицей ключей (вывод, сортировка).	Сообщение об ошибке: Таблица ключей пуста
12	Попытка создания таблицы ключей для пустой таблицы.	Сообщение об ошибке: Пустая таблица
13	Попытка просмотра по ключам без создания таблицы ключей.	Сообщение об ошибке: Сначала создайте таблицу ключей.
14	Попытка ввода пустого названия театра.	Заново ввести название театра
15	Попытка ввода пустого названия спектакля.	Заново ввести название спектакля

Оценка эффективности

1. Записи хранятся в одном массиве, при этом для каждого типа спектакля используются только релевантные поля благодаря union.
2. Прямой доступ к полям записи без необходимости вычисления сложных смещений.
3. Сортировка ключей выполняется быстрее, так как перемещаются только небольшие структуры.
4. Можно создать несколько таблиц ключей для разных критериев сортировки.

5. Обеспечивается быстрый доступ к записям по заданному критерию.

Размер	Быстрая сортировка данных	Быстрая сортировка ключей	Пузырьковая сортировка данных	Пузырьковая сортировка ключей
50	0.45			
100				
200				
500				

Вывод

Для того чтобы эффективно управлять данными о театральных спектаклях с различными характеристиками, была применена структура данных с вариантной частью, реализованная через объединение (union) в языке С. Это позволило оптимально использовать память, храня только соответствующие поля для каждого типа спектакля - пьесы или музыкального представления.

Реализация программы потребовала создания гибкой системы обработки таблиц, включающей как работу непосредственно с основной таблицей записей, так и использование дополнительной таблицы ключей для ускорения операций сортировки. Сравнительный анализ алгоритмов сортировки показал значительное преимущество быстрой сортировки перед пузырьковой, особенно при работе с большими объемами данных. Использование таблицы ключей продемонстрировало свою эффективность за счет сокращения времени выполнения операций и экономии памяти при частых операциях сортировки.

Контрольные вопросы

1. Как выделяется память под вариантную часть записи?

Память под вариантную часть записи выделяется следующим образом:

- Объем памяти для записи с вариантами складывается из длин полей фиксированной части и максимального по длине поля вариантной части
- В языке С вариантные записи реализуются структурами с объединениями (union)
- Поля записи занимают в памяти непрерывную область
- При компиляции программы вычисляется смещение каждого поля относительно начала записи
- Для вариантной части выделяется память, достаточная для хранения самого большого из возможных вариантов структур

2. Что будет, если в вариантную часть ввести данные, несоответствующие описанным?

При вводе данных, несоответствующих описанным в вариантной части:

- Тип данных в вариантной части при компиляции не проверяется
- Контроль за правильностью использования вариантной части возлагается на программиста

- Может произойти некорректное поведение программы или ошибки времени выполнения
- Данные могут интерпретироваться неправильно, что приведет к искажению информации
- Возможны ошибки доступа к памяти и неопределенное поведение программы

3. Кто должен следить за правильностью выполнения операций с вариантной частью записи?

Ответственность за правильность выполнения операций с вариантной частью записи лежит на:
- Программисте - он должен обеспечить корректную работу с вариантной частью
- Необходимо реализовать проверки соответствия данных перед доступом к полям вариантной части
- Программист должен следить за тем, чтобы обращение к полям вариантной части соответствовало текущему активному варианту
- Требуется обеспечить валидацию вводимых данных и обработку ошибочных ситуаций

4. Что представляет собой таблица ключей, зачем она нужна?

Таблица ключей представляет собой:
- Дополнительный массив, содержащий индекс элемента в исходной таблице и выбранный ключ
- Ключ - это поле записи, по которому осуществляется поиск или сортировка

Таблица ключей нужна для:
- Ускорения операций сортировки (перемещаются только ключи, а не вся структура)
- Экономии времени при обработке больших таблиц
- Сохранения исходного порядка данных в основной таблице
- Возможности иметь несколько различных представлений одних и тех же данных
- Снижения затрат на структурирование информации

5. В каких случаях эффективнее обрабатывать данные в самой таблице, а когда – использовать таблицу ключей?

Обработка в самой таблице эффективнее когда:
- Размер таблицы небольшой
- Записи имеют малый размер
- Требуется частый доступ к разным полям записей
- Производится мало операций сортировки
- Важно сохранить физический порядок записей

Использование таблицы ключей эффективнее когда:
- Таблица имеет большой размер
- Записи содержат много полей и имеют значительный размер
- Часто выполняются операции сортировки по одному полю
- Требуется несколько различных представлений данных
- Важно минимизировать перемещения данных в памяти
- Необходимо сохранить исходный порядок записей

6. Какие способы сортировки предпочтительнее для обработки таблиц и почему?

Для обработки таблиц предпочтительнее:

Быстрая сортировка (ускоренные алгоритмы):
- Имеет среднюю временную сложность O(n log n)
- Эффективна для больших объемов данных
- Требует меньше операций сравнения и перестановок
- Особенно эффективна при использовании с таблицей ключей

Пузырьковая сортировка (простые алгоритмы):
- Проста в реализации и понимании
- Может быть эффективна для очень маленьких массивов
- Полезна для учебных целей
- Стабильна (сохраняет порядок равных элементов)

Выбор алгоритма зависит от:
- Размера таблицы
- Частоты операций сортировки
- Требований к производительности
- Необходимости стабильности сортировки

Для больших таблиц рекомендуется использовать быструю сортировку в сочетании с таблицей ключей для достижения максимальной эффективности.